

# 网络技术与应用实验报告

## 实验4：互联网组网与路由器配置

姓名：谢小珂 学号：2310422 专业：物联网工程

### 一、实验要求

#### 1.实体环境下互联网组网与路由器配置

在实体环境下完成互联网组网与路由器配置，要求如下：

- (1) 在机房实验室环境下，通过将局域网划分为不同子网，用多IP主机作为路由器，组建互联网。
- (2) 在命令行方式下，按照静态路由方式，配置路由器和主机，测试互联网的连通性。

#### 2.仿真环境下的互联网组网与路由器配置

在仿真环境下完成互联网组网与路由器配置，要求如下：

- (1) 学习路由器的配置方法和配置命令。
- (2) 参考实体实验，组建由多个路由器组成的互联网。物理网络可以由集线器、交换机构成。
- (3) 按照静态路由方式配置路由器和主机，测试互联网的连通性。
- (4) 利用动态路由方式配置路由器和主机，测试互联网的连通性。
- (5) 在仿真环境的“模拟”方式中观察数据包在互联网中的传递过程，并进行分析。

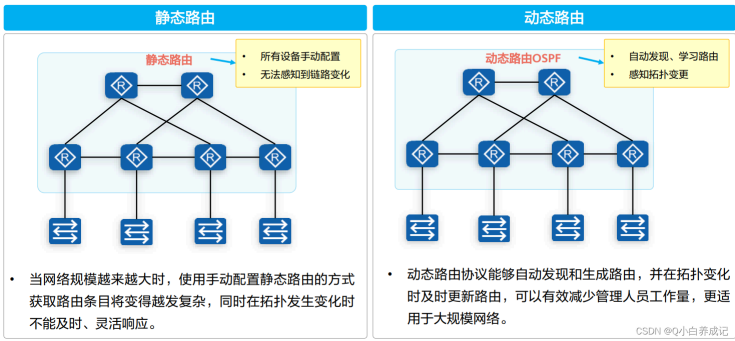
### 二、实验原理

#### 2.1 多子网互联与路由器转发原理

IP 网络通过在不同物理或逻辑链路上划分多个子网，每个子网用“网络地址 + 子网掩码（前缀）”标识。路由器（或开启 IP 转发的多 IP 主机）在各子网之间转发分组：对到达接口的数据报，根据目的 IP 地址在路由表中进行最长前缀匹配，找到匹配条目中给出的下一跳地址或出口接口，将数据报转发出去，从而实现多子网的互联。

#### 2.2 静态路由与 RIP/OSPF 动态路由

静态路由由管理员在路由器上手工配置，例如 `ip route 192.168.3.0 255.255.255.0 10.0.12.2`，直接指定到某目的网络的下一跳或出口，结构简单、开销小，适合本实验这种拓扑固定、规模较小的环境。



动态路由通过路由协议自动维护路由表。本实验实际采用 RIP，RIP 属于距离向量型动态路由协议，以“跳数”为度量，路由器周期性向邻居通告自己的路由表，根据收到的更新选择代价最小的路径并自动生成以 R 标记的路由表项，实现路由的自动学习和收敛。与之对应，OSPF 是典型的链路状态型动态路由协议，各路由器周期性发布链路状态通告（LSA），在全网形成统一的拓扑视图，并利用最短路径优先（SPF）算法按“代价”计算到各网段的最优路径，适用于规模更大、结构更复杂的 IP 网络。

## 2.3 基于 ICMP 的端到端连通性验证

ping 命令基于 ICMP 回显请求/应答报文工作：源主机向目的 IP 发送 ICMP Echo Request，沿路径上的路由器只根据路由表转发 IP 数据报，不修改 ICMP 负载；当目的主机收到请求后返回 Echo Reply，源主机据此判断端到端连通性和往返时延。因此，在本实验中只有当各主机 IP、子网掩码和默认网关配置正确，且各路由器静态/动态路由表一致时，ping 才能成功，能够直观反映组网与路由配置是否正确。

# 三、实验过程

## 3.1 实体环境下互联网组网与路由器配置

### 3.1.1 实验拓扑与地址规划

机房提供两台 Windows 主机，通过交换机处于同一以太网内，本实验选用其中一台作为多 IP 主机路由器（记为 A），另一台作为普通主机（记为 B），在同一物理网络上划分出两个逻辑子网，实现跨子网互联。

地址规划如下：

- 主机 A（多 IP 主机 / 路由器）
  - 网卡“本地连接2”：
    - IPv4 地址 1：192.168.10.1 / 255.255.255.0
    - IPv4 地址 2：192.168.20.1 / 255.255.255.0
- 主机 B（普通主机）
  - 网卡“本地连接”：192.168.10.2 / 255.255.255.0
  - 默认网关：后续通过静态路由显式指定，不在 TCP/IP 属性中配置

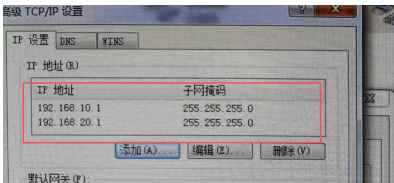
这样，逻辑上 A 同时连接到两个子网 192.168.10.0/24 和 192.168.20.0/24，相当于一台具有两个三层接口的路由器；B 仅位于 192.168.10.0/24 子网。

### 3.1.2 主机 A 配置多 IP 与 IP 转发

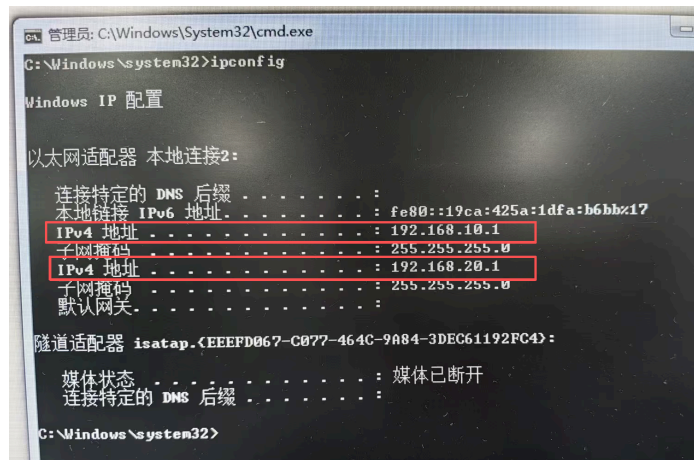
#### 1. 多 IP 地址配置

在主机 A 上，进入“网络和共享中心 → 更改适配器设置”，右键“本地连接2”，在 IPv4 属性中添加两个静态地址：

- IP1：192.168.10.1，子网掩码 255.255.255.0；
- IP2：192.168.20.1，子网掩码 255.255.255.0。



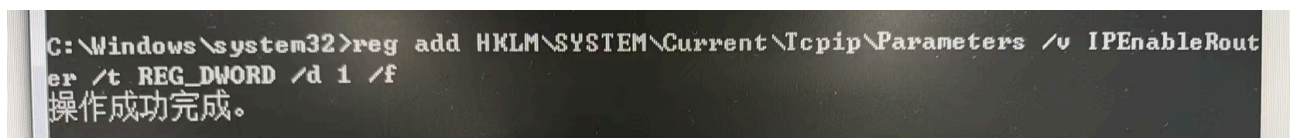
- 通过 `ipconfig` 命令确认网卡上确实存在两条 IPv4 地址记录。



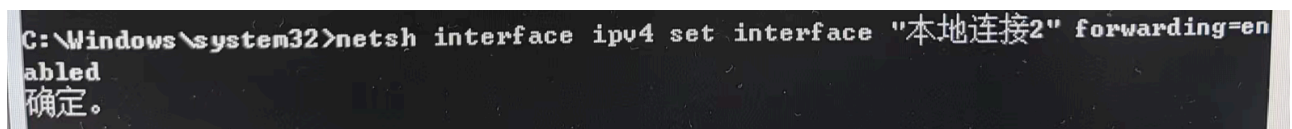
## 2. 开启 IP 转发功能

打开“命令提示符（管理员）”，执行以下命令启用系统路由功能并允许网卡转发：

```
1 reg add HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters ^
2     /v IPEnableRouter /t REG_DWORD /d 1 /f
3
4 netsh interface ipv4 show interfaces ← 查看接口名称和索引
5 netsh interface ipv4 set interface "本地连接2" forwarding=enabled
```



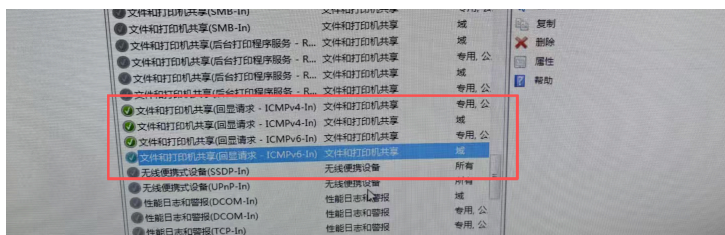
第一条命令在注册表中将 `IPEnableRouter` 置为 1，开启 IP 层转发。



第二条命令在 IPv4 配置中将“本地连接2”接口的转发属性设为 enabled，使该接口可以转发经过的数据报。

## 3. 允许被 ping（ICMP 入站规则）

在“高级安全 Windows 防火墙”中，启用“文件和打印机共享(回显请求-ICMPv4-In)”相关入站规则，保证主机 A 能够响应来自 B 的 ICMP Echo 请求，便于后续连通性测试。



### 3.1.3 主机 B 配置 IP 与静态路由

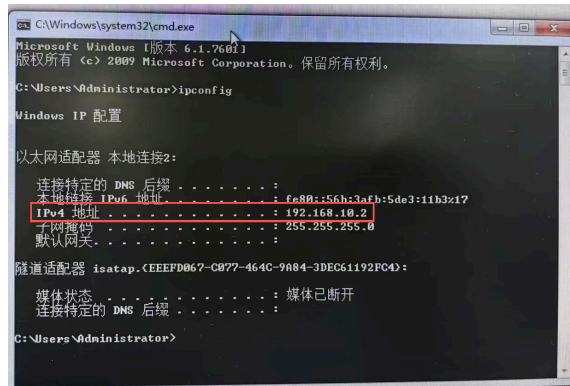
#### 1. IP 地址配置

在主机 B 上，将本地连接配置为：

- IPv4 地址：192.168.10.2
- 子网掩码：255.255.255.0

- 默认网关：留空

使用 `ipconfig` 检查配置是否生效。



## 2. 添加到 192.168.20.0/24 的静态路由

在主机 B 的“命令提示符（管理员）”中，添加一条永久静态路由：

```
1 | route -p add 192.168.20.0 mask 255.255.255.0 192.168.10.1
```

该命令表示：到达网络 192.168.20.0/24 的报文，下一跳交给 192.168.10.1，即多 IP 主机 A。

通过 `route print` 验证路由表中出现相应的“永久路由”条目：

```
1 | 192.168.20.0    255.255.255.0    192.168.10.1    ...
```

### 3.1.4 连通性测试与结果

#### 1. 同一子网连通性测试

在主机 B 上执行：

```
1 | ping 192.168.10.1
```

结果收到连续的 Echo Reply，说明 B 与 A 在 192.168.10.0/24 子网内二层、三层通信正常。

#### 2. 跨子网连通性测试

继续在主机 B 上执行：

```
1 | ping 192.168.20.1
```

此时 ICMP 报文按照静态路由条目，先从 B 发送到 192.168.10.1，再由开启 IP 转发的多 IP 主机 A 在两个子网之间转发到 192.168.20.1。实验结果显示能够正常收到 Echo Reply，表明：

- 静态路由配置正确，B 能够将去往 192.168.20.0/24 的流量交给 A；
- 多 IP 主机 A 的路由转发功能生效，成功在两个逻辑子网之间转发数据报。

```
C:\Windows\system32>ping 192.168.20.1

正在 Ping 192.168.20.1 具有 32 字节的数据:
来自 192.168.20.1 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.20.1 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.20.1 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.20.1 的回复: 字节=32 时间<1ms TTL=128

192.168.20.1 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 0ms, 平均 = 0ms

C:\Windows\system32>ping 192.168.10.1

正在 Ping 192.168.10.1 具有 32 字节的数据:
来自 192.168.10.1 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.10.1 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.10.1 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.10.1 的回复: 字节=32 时间<1ms TTL=128

192.168.10.1 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 0ms, 平均 = 0ms

C:\Windows\system32>
```

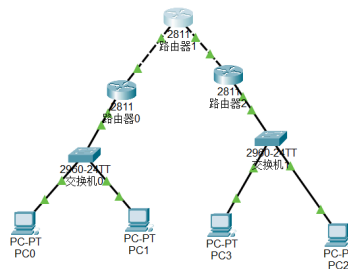
如图，实体环境下“将局域网划分为不同子网、以多 IP 主机代替路由器、通过命令行配置静态路由实现互联网连通性”的实验目标完成。

### 3.2 仿真环境下的互联网组网与路由器配置

#### 3.2.1 拓扑搭建与地址规划

在 Cisco Packet Tracer 中搭建如下拓扑：

- 三台路由器：R1、R2、R3（2811）；
- 两台二层交换机：SW1、SW2（2960-24TT）；
- 四台主机：PC0、PC1 连接左侧交换机，PC2、PC3 连接右侧交换机。



地址规划与实体实验保持一致的“左右两个局域网 + 中间两条点到点链路”结构：

- 左侧局域网：192.168.1.0/24
  - PC0：192.168.1.10 /24，网关 192.168.1.1
  - PC1：192.168.1.20 /24，网关 192.168.1.1
  - R1 F0/0：192.168.1.1 /24
- 右侧局域网：192.168.3.0/24
  - PC2：192.168.3.10 /24，网关 192.168.3.1
  - PC3：192.168.3.20 /24，网关 192.168.3.1
  - R3 F0/0：192.168.3.1 /24
- 路由器之间点到点链路：
  - R1 F0/1 – R2 F0/0：10.0.12.0/30

- R1 F0/1: 10.0.12.1 /30
- R2 F0/0: 10.0.12.2 /30
- R2 F0/1 – R3 F0/1: 10.0.23.0/30
- R2 F0/1: 10.0.23.1 /30
- R3 F0/1: 10.0.23.2 /30

### 3.2.2 路由器基础配置（命令行方式）

本实验所有路由器均通过命令行（CLI）配置。以 R1、R2、R3 为例，分别完成**进入配置模式** → **设置主机名** → **配置各接口 IP 地址** → **打开接口** → **保存配置** → **验证状态**等步骤。

#### （1）R1 的基础配置及命令说明

R1 负责连接左侧局域网（192.168.1.0/24）和中间链路 10.0.12.0/30。其完整配置如下：

```
1 Router(config)#hostname R1
2 R1(config)#interface FastEthernet0/0
3 R1(config-if)#ip address 192.168.1.1 255.255.255.0
4 R1(config-if)#no shutdown
5 R1(config-if)#exit
6 R1(config)#interface FastEthernet0/1
7 R1(config-if)#ip address 10.0.12.1 255.255.255.252
8 R1(config-if)#no shutdown
9 R1(config-if)#exit
10 R1(config)#end
11 R1#write
```

逐行说明如下：

- **hostname R1**  
将路由器主机名由默认的 Router 改为 R1，便于在多台设备环境中区分，后续提示符也会变成 R1(config)#。
- **interface FastEthernet0/0**  
进入接口 F0/0 的接口配置模式 R1(config-if)#，后续 IP、启用/关闭等操作都作用在该接口上。
- **ip address 192.168.1.1 255.255.255.0**  
给 F0/0 配置 IP 地址为 192.168.1.1，子网掩码 /24，作为左侧局域网 192.168.1.0/24 的网关地址，与 PC0、PC1 的网关配置保持一致。
- **no shutdown**  
将接口从默认的 administratively down 状态启动为 up。Cisco 路由器接口默认是关闭的，若不执行该命令，接口不会转发任何数据。
- **exit**  
从接口配置模式 R1(config-if)# 退出回到全局配置模式 R1(config)#，以便继续配置其他接口。
- **interface FastEthernet0/1**  
进入 F0/1 接口配置模式，用于连接中间路由器 R2。

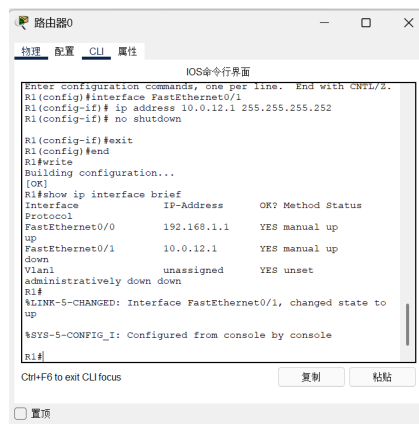


- `ip address 10.0.12.1 255.255.255.252`  
为 R1 与 R2 的点到点链路配置 IP。这里选用 `/30` 掩码，只保留 2 个可用地址（10.0.12.1 与 10.0.12.2），正好对应链路两端，既节省地址空间又简单清晰。
  - `no shutdown`  
同样打开该接口，使其进入 up 状态，保证 R1 与 R2 之间链路可用。
  - `end`  
退出配置模式，返回特权模式 `R1#`，表示当前配置阶段结束。
  - `write`（或 `copy running-config startup-config`）  
将当前运行配置写入 NVRAM 的启动配置中，防止设备重启后配置丢失，是实际环境中非常重要的一步。
- 通过以下命令进行验证：

```
1 R1#show ip interface brief
```

该命令简要列出各接口的 IP 地址和状态：

- `IP-Address` 一列显示接口 IP 是否配置正确；
- `Status` / `Protocol` 两列应为 `up/up`，表示链路物理层和数据链路层状态均正常；
- 如某接口仍为 `administratively down` 或 `down`，需要重新检查是否执行了 `no shutdown` 命令以及物理连接是否正确。



## (2) R2 的基础配置及命令说明

R2 位于中间，连接 R1 和 R3，两侧链路分别为 10.0.12.0/30 与 10.0.23.0/30。配置如下：

```

1 Router>enable
2 Router#configure terminal
3 Router(config)#hostname R2
4 R2(config)#interface FastEthernet0/0
5 R2(config-if)#ip address 10.0.12.2 255.255.255.252
6 R2(config-if)#no shutdown
7 R2(config-if)#exit
8 R2(config)#interface FastEthernet0/1
9 R2(config-if)#ip address 10.0.23.1 255.255.255.252
10 R2(config-if)#no shutdown
11 R2(config-if)#exit
12 R2(config)#end
13 R2#write

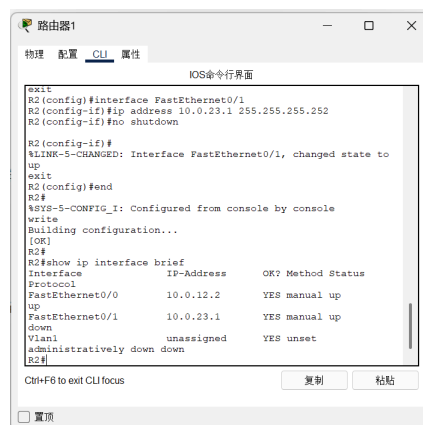
```

关键命令含义与 R1 相同，不同之处在于 IP 地址的角色：

- `ip address 10.0.12.2 255.255.255.252`  
将 R2 F0/0 配置为 10.0.12.2，与 R1 F0/1 的 10.0.12.1 组成点到点网，两端必须在同一网段才能互通。
- `ip address 10.0.23.1 255.255.255.252`  
将 R2 F0/1 配置为 10.0.23.1，用于与 R3 的 F0/1（10.0.23.2）相连，同样构成一个 /30 点到点链路。R2 作为中间路由器，将来会通过这两个接口在左右网络之间转发数据。
- 通过以下命令进行验证：

```
1 R2#show ip interface brief
```

如图：



### (3) R3 的基础配置及命令说明

R3 连接右侧局域网（192.168.3.0/24）和中间链路 10.0.23.0/30。配置如下：



```

1 Router>enable
2 Router#configure terminal
3 Router(config)#hostname R3
4 R3(config)#interface FastEthernet0/0
5 R3(config-if)#ip address 192.168.3.1 255.255.255.0
6 R3(config-if)#no shutdown
7 R3(config-if)#exit
8 R3(config)#interface FastEthernet0/1
9 R3(config-if)#ip address 10.0.23.2 255.255.255.252
10 R3(config-if)#no shutdown
11 R3(config-if)#exit
12 R3(config)#end
13 R3#write

```

说明：

- `ip address 192.168.3.1 255.255.255.0`  
将 F0/0 配置为右侧局域网的网关地址，PC2、PC3 的默认网关都指向该地址，所有出网流量会先发给 R3。
- `ip address 10.0.23.2 255.255.255.252`  
为 R3 与 R2 的点到点链路另一端地址，与 R2 F0/1 的 10.0.23.1 同网段。

其余命令（`hostname`、`no shutdown`、`end`、`write`）作用与 R1/R2 相同，分别用于命名、启用接口、退出配置和保存当前配置。

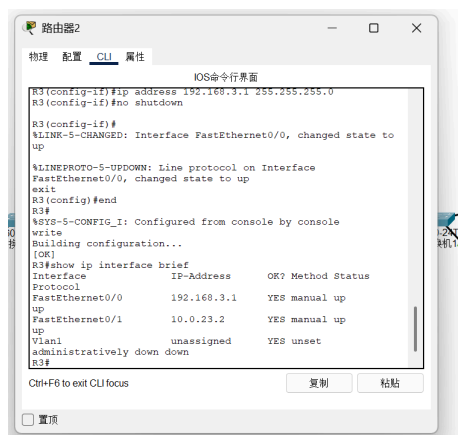
通过以下命令进行验证：

```

1 R3#show ip interface brief

```

如图：



通过以上步骤，三台路由器的基础三层配置完成，为后续静态路由与 RIP/OSPF 动态路由配置奠定了正确的 IP 和接口连通环境。

### 3.2.3 静态路由配置与测试

在完成三台路由器接口 IP 配置并确认各直连网段可达的基础上，需要通过**静态路由**让路由器“认识”非直连网段，实现端到端互通。本实验的目标是使左右两个局域网 `192.168.1.0/24` 与 `192.168.3.0/24` 通过中间两条点到点链路互通。

## (1) R1 上静态路由配置及解释

在 R1 上进入全局配置模式，输入：

```
1 R1#configure terminal
2 R1(config)#ip route 192.168.3.0 255.255.255.0 10.0.12.2
3 R1(config)#end
4 R1#write
```

命令含义：

- `ip route 192.168.3.0 255.255.255.0 10.0.12.2`

这是一个典型的静态路由命令，格式为：

```
1 ip route <目的网络> <子网掩码> <下一跳地址>
```

具体到本实验：

- 目的网络：192.168.3.0
- 掩码：255.255.255.0（即 /24）
- 下一跳：10.0.12.2（R2 在 R1-R2 链路一侧的 IP）

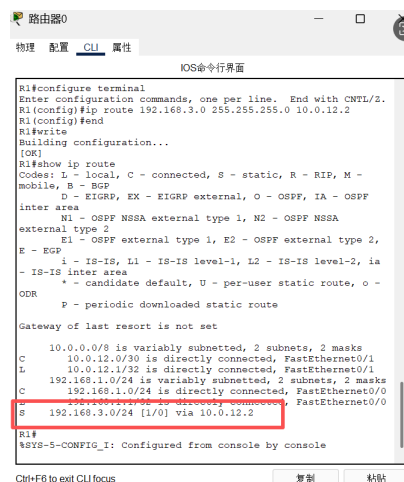
含义是：当 R1 收到去往 192.168.3.0/24 的数据报时，将其交给下一跳 10.0.12.2 转发。由于 R1 与 10.0.12.2 是直连关系，R1 会把报文发给 R2，由 R2 继续转发到右侧。

- `end`：退出配置模式回到特权模式。
- `write`：将新增的静态路由配置保存到启动配置中。

配置完成后，用：

```
1 R1#show ip route
```

可以看到路由表中多了一条以 `S` 标记的路由：



```
1 S 192.168.3.0/24 [1/0] via 10.0.12.2
```

其中 `S` 表示静态路由，`via 10.0.12.2` 表示下一跳。

## (2) R2 上静态路由配置及解释

R2 作为中间路由器，需要知道左右两个 LAN 的去向。配置如下：

```
1 R2#configure terminal
2 R2(config)#ip route 192.168.1.0 255.255.255.0 10.0.12.1
3 R2(config)#ip route 192.168.3.0 255.255.255.0 10.0.23.2
4 R2(config)#end
5 R2#write
```

解释：

- `ip route 192.168.1.0 255.255.255.0 10.0.12.1`

表示：去往左侧 LAN 192.168.1.0/24 的报文，一律发给 10.0.12.1（R1 在这条链路一端的地址）。  
因为 R2 与 R1 在 10.0.12.0/30 子网直连，R2 只需把包交给 R1，由 R1 再发给局域网即可。

- `ip route 192.168.3.0 255.255.255.0 10.0.23.2`

表示：去往右侧 LAN 192.168.3.0/24 的报文，一律发给 10.0.23.2（R3 在 R2-R3 链路上的地址）。  
R3 直连 192.168.3.0/24，因此 R2 不用知道内部结构，只要把包交给 R3。

执行 `show ip route`，可以看到：



```
1 S 192.168.1.0/24 [1/0] via 10.0.12.1
2 S 192.168.3.0/24 [1/0] via 10.0.23.2
```

说明 R2 已经有了两条指向左右两端局域网的静态路由，能够作为中间转发节点使用。

## (3) R3 上静态路由配置及解释

R3 接入右侧 LAN，需要知道如何返回左侧网络。配置如下：

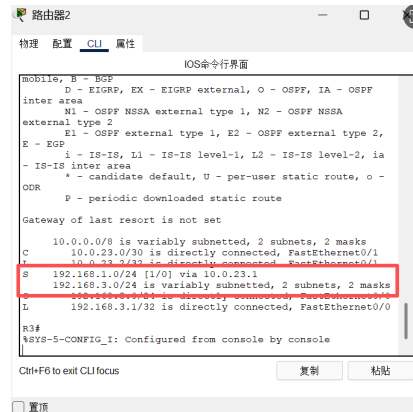
```
1 R3#configure terminal
2 R3(config)#ip route 192.168.1.0 255.255.255.0 10.0.23.1
3 R3(config)#end
4 R3#write
```

说明：

- `ip route 192.168.1.0 255.255.255.0 10.0.23.1`

告诉 R3：任何发往 192.168.1.0/24 的报文，下一跳是 10.0.23.1（R2 的 F0/1 接口）。  
R3 将报文交给 R2，再由 R2 按上一步配置的静态路由转发到 R1，最终到达左侧 LAN。

用 `show ip route` 可以看到：



```
1 | s 192.168.1.0/24 [1/0] via 10.0.23.1
```

至此，三台路由器的静态路由配置完成，所有“非直连”网段在路由表中都已经有了明确的下一跳。

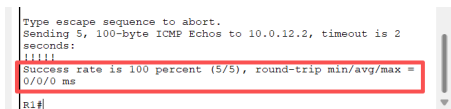
#### (4) 静态路由连通性测试与分析

##### 1. 局部测试：路由器之间的互通

在配置路由前，先使用 `ping` 验证路由器之间的直连链路是否正常，例如：

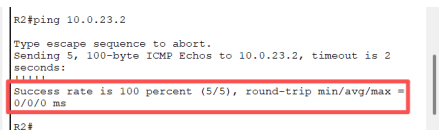
- 在 R1 上：

```
1 | R1#ping 10.0.12.2
```



- 在 R2 上：

```
1 | R2#ping 10.0.23.2
```



均能成功，说明物理链路和接口 IP 配置无误，可以开始测试端到端。

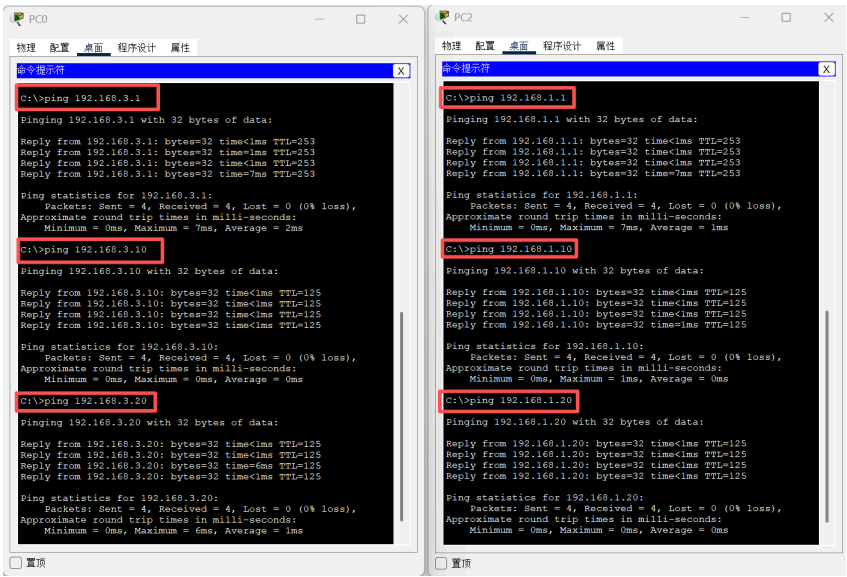
##### 1. 端到端连通性测试（关键）

在 PC0（左侧 LAN）上：

```
1 | ping 192.168.3.1    ← R3 的 LAN 接口
2 | ping 192.168.3.10  ← PC2
3 | ping 192.168.3.20  ← PC3
```

在 PC2（右侧 LAN）上：

```
1 | ping 192.168.1.1    ← R1 的 LAN 接口
2 | ping 192.168.1.10  ← PC0
3 | ping 192.168.1.20  ← PC1
```



测试现象与分析：

- 能够从左侧 PC ping 通右侧路由器和 PC，表明报文路径为：  
PC0 → R1 → R2 → R3 → PC2/PC3，路由表配置正确。
- 能够从右侧 PC ping 回左侧 PC，表明返回路径  
PC2 → R3 → R2 → R1 → PC0 同样根据静态路由正确转发，实现了双向可达。
- 实验中偶尔第一次 ping 会出现 1 次 Request timed out，后续 3 次成功，这是因为第一次需要进行 ARP 解析，延时略大，属于正常现象。

3.2.4 RIP 动态路由配置与测试

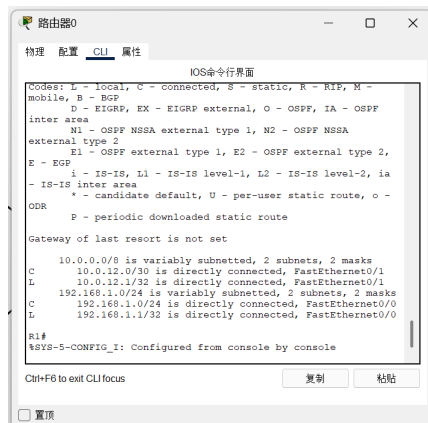
在静态路由实验的基础上，本小节改为使用 RIP 动态路由协议，让三台路由器自动交换路由信息、生成路由表，从而在不依赖手工 ip route 的情况下实现端到端互通。

(1) 删除静态路由，避免干扰

在启用动态路由之前，需要先将 3.2.3 中配置的静态路由删除，否则静态路由由优先级较高，可能遮挡 RIP 学到的路由，使实验结果不直观。

以 R1 为例，删除去往 192.168.3.0/24 的静态路由：

```
1 | R1#configure terminal
2 | R1(config)#no ip route 192.168.3.0 255.255.255.0 10.0.12.2
3 | R1(config)#end
```



执行完毕后，在路由器上用：

```
1 | R1#show ip route
```

确认路由表中不再有以 **S** 标记的静态路由，只保留各自的直连网段（C、L）。至此，路由环境“清空”为纯直连状态，便于观察 RIP 的学习过程。

## (2) 在三台路由器上启用 RIPv2

RIP 属于距离向量路由协议，按“跳数”作为度量（metric），路由器定期向邻居广播自己的路由表。实验中采用 RIPv2，支持无类域间路由和子网掩码传递。

**R1 上的 RIP 配置：**

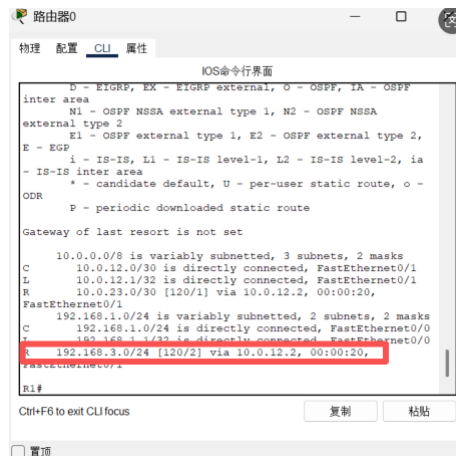
```
1 | R1#configure terminal
2 | R1(config)#router rip
3 | R1(config-router)#version 2
4 | R1(config-router)#no auto-summary
5 | R1(config-router)#network 192.168.1.0
6 | R1(config-router)#network 10.0.12.0
7 | R1(config-router)#end
8 | R1#write
```

命令说明：

- **router rip**  
启动 RIP 路由进程，并进入 **R1(config-router)#** 模式。在该模式下可以设置 RIP 版本、通告的网络等。
- **version 2**  
指定使用 RIPv2。相对于 RIPv1，RIPv2 支持 VLSM/CIDR，能够携带子网掩码信息，适用于本实验存在多种掩码（/24 和 /30）的环境。
- **no auto-summary**  
关闭自动汇总功能。默认情况下，RIP 会按“类ful边界”对路由进行自动汇总，可能导致前缀不精确；关闭后，路由器会按实际子网前缀通告，更符合当前精细子网划分的需求。
- **network 192.168.1.0**  
告诉 R1：所有 IP 地址落在 192.168.1.0/24 网段的接口（即 F0/0）要启用 RIP：
  - 在该接口上向邻居发送 RIP 路由更新；
  - 在该接口上接收来自邻居的 RIP 更新；

- 将该直连网络 192.168.1.0/24 通告给其他路由器。
- `network 10.0.12.0`  
同理，启用 R1 在 10.0.12.0/30 链路上的 RIP 功能，使其能与 R2 通过此链路交换路由信息。
- `write`  
将 RIP 配置保存到启动配置文件中。
- RIP 配置完成后，等待若干个更新周期（通常几个秒到几十秒），在路由器上执行：

```
1 R1#show ip route
```



可以观察到：

- 在 R1 的路由表中，除了 C、L 直连网段外，新增一条以 R 标记的路由：

```
1 R 192.168.3.0/24 [120/2] via 10.0.12.2, FastEthernet0/1
```

表示：

- 路由来源为 RIP（R）；
- 管理距离 120，度量值（metric）为 2 跳（R1 → R2 → R3）；
- 下一跳为 10.0.12.2，即 R2。

## R2 上的 RIP 配置：

```
1 R2#configure terminal
2 R2(config)#router rip
3 R2(config-router)#version 2
4 R2(config-router)#no auto-summary
5 R2(config-router)#network 10.0.12.0
6 R2(config-router)#network 10.0.23.0
7 R2(config-router)#end
8 R2#write
```

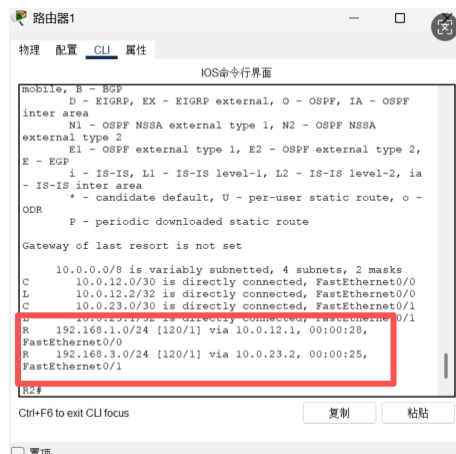
说明：

- R2 作为中间路由器，它自己不直接连接任何 LAN，只连接两个点到点子网 10.0.12.0/30 和 10.0.23.0/30。
- 通过 `network 10.0.12.0` 和 `network 10.0.23.0`，R2 会在这两条链路上启用 RIP：



- 从 R1 接收关于 192.168.1.0/24 的路由；
- 从 R3 接收关于 192.168.3.0/24 的路由；
- 并将各自学到的路由再通告给对端，实现“左 LAN ↔ 右 LAN”的信息传播。
- 配置完成后，在路由器上执行：

```
1 | R2#show ip route
```



在 R2 的路由表中，会同时有两条 RIP 路由：

```
1 | R 192.168.1.0/24 [120/1] via 10.0.12.1, FastEthernet0/0
2 | R 192.168.3.0/24 [120/1] via 10.0.23.2, FastEthernet0/1
```

说明 R2 距离左右两端 LAN 的跳数均为 1，是典型的“中间转发节点”。

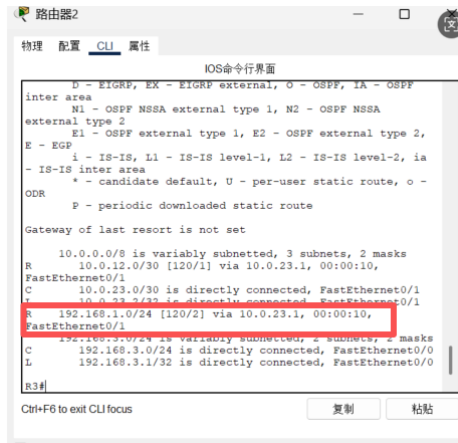
### R3 上的 RIP 配置：

```
1 | R3#configure terminal
2 | R3(config)#router rip
3 | R3(config-router)#version 2
4 | R3(config-router)#no auto-summary
5 | R3(config-router)#network 10.0.23.0
6 | R3(config-router)#network 192.168.3.0
7 | R3(config-router)#end
8 | R3#write
```

说明：

- `network 10.0.23.0`：在与 R2 相连的链路上启用 RIP，用于收发路由更新。
- `network 192.168.3.0`：使 R3 把右侧局域网 192.168.3.0/24 加入 RIP 路由域，并作为可达网段通告给其他路由器。
- 配置完成后，在路由器上执行：

```
1 | R3#show ip route
```



出现对应的：

1 R 192.168.1.0/24 [120/2] via 10.0.23.1, FastEthernet0/1

### (3) 基于 RIP 的端到端连通性测试

在静态路由全部删除、仅启用 RIP 的前提下，重新进行端到端 ping 测试。

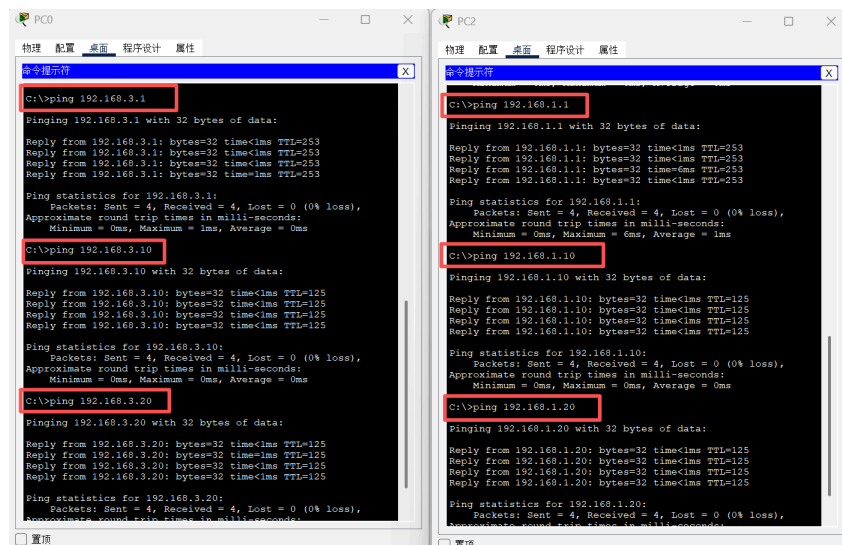
左侧局域网 → 右侧局域网：

在 PC0 上：

```
1 ping 192.168.3.1 ← R3 的 LAN 接口
2 ping 192.168.3.10 ← PC2
3 ping 192.168.3.20 ← PC3
```

现象：

- ICMP 报文从 PC0 出发，经 SW1 到 R1，R1 通过 RIP 路由表选择下一跳 10.0.12.2，将报文送至 R2；
- R2 再根据 RIP 路由表选择下一跳 10.0.23.2，将报文送至 R3；
- R3 将报文转发到右侧 LAN，最终到达 PC2/PC3，并返回 Echo Reply。



右侧局域网 → 左侧局域网：

在 PC2 或 PC3 上：

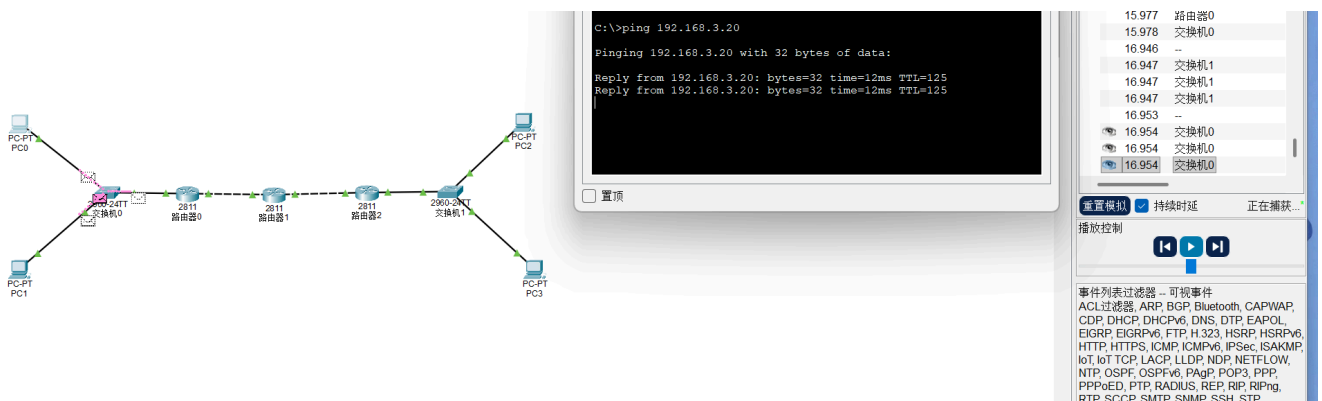
```
1 | ping 192.168.1.1    ← R1 的 LAN 接口
2 | ping 192.168.1.10  ← PC0
3 | ping 192.168.1.20  ← PC1
```

返回路径按照 RIP 计算出的最短路径 **R3 → R2 → R1** 逐跳返回。

可以验证：在取消所有静态路由后，仅依靠 RIP 动态路由协议，三台路由器能够自动发现并维护到远端局域网的路由，使整个仿真互联网在拓扑不变的情况下具备与静态路由实验相同的端到端可达性。这为后续对比不同动态路由协议（如 RIP 与 OSPF）的收敛特性和适用场景提供了基础。

### 3.2.5 Simulation 模式观察报文转发过程

在 Packet Tracer 中切换到 **Simulation** 模式，选择 ICMP 协议，从 PC0 向 PC3 发送 **ping**。通过“捕获/前进”按钮逐步查看报文在拓扑中的传递过程，可以观察到：



1. Echo Request 报文首先从 PC0 发出，经 SW1 转发到 R1；
2. R1 按路由表将报文从 F0/1 发往 R2，R2 再转发至 R3；
3. R3 将报文送到右侧局域网，经 SW2 到达 PC3；
4. PC3 返回 Echo Reply，沿相反路径逐跳返回 PC0。

在每一跳中 IP 源地址和目的地址保持不变，TTL 逐跳减 1，而二层源/目的 MAC 地址随链路变化而改变，这与路由器三层转发、二层重封装的理论过程一致。

通过上述仿真实验，完成了路由器 CLI 配置方法的学习，对比了静态路由和 RIP 动态路由的行为，并利用 Simulation 模式直观理解了数据包在多路由器互联网中的转发过程。

## 四、实验总结

### 1. 遇到的问题及解决方法

- 仿真环境中，初期静态路由和 RIP 配置完成后，端到端 **ping** 偶尔不通或首包超时，主要原因是接口未 **no shutdown**、网段写错或还在 ARP 学习阶段。通过逐台路由器检查 **show ip interface brief**、**show ip route**，核对 IP 地址、掩码和下一跳配置，并在必要时反复 **ping** 直连对端接口，最终确认链路状态和路由表均正确，连通性恢复正常。
- 实体环境中，机房主机之间最初无法互相 **ping**，原因包括：多 IP 主机未开启 IP 转发（**IPEnableRouter** 未置 1）、未在 B 机上添加静态路由以及 Windows 防火墙默认禁止 ICMP 回显请求。通过修改注册表并使用 **netsh** 打开接口转发、在 B 机上使用 **route -p add** 添加到 192.168.20.0/24 的静态路由，同时在防火墙中启用 ICMP 入站规则，最终实现了跨子网 **ping** 通，完成实体组网要求。

## 2. 实验收获与体会

- 理解了路由器和多 IP 主机在网络层的本质作用：都依赖路由表进行“下一跳转发”，静态路由由管理员显式配置，RIP/OSPF 等动态路由协议则自动维护路由表。
- 通过 Packet Tracer 实验，熟悉了路由器接口配置、静态路由与 RIP 动态路由的基本命令，并利用 Simulation 模式直观观察了数据包逐跳转发的过程。
- 通过机房实体实验，将“多 IP 主机 + 静态路由 + IP 转发 + 防火墙配置”串联起来，体会到理论配置、系统实现和故障排查（三层路由、二层连通、主机安全策略）之间的紧密关系。